
LAKEQA: An Exploratory QA Benchmark over a Million-Scale Data Lake

Abstract

Recent large language models (LLMs) have shown rapid progress on reading-based question answering (QA), where the evidence is explicitly provided or trivially retrievable. In contrast, real-world questions are often not paired with accurate evidence documents. The useful evidence resides in a massive collection of data lakes, necessitating searching as a prerequisite for answering. However, there is a lack of a comprehensive benchmark that requires searching and reasoning over a large collection of data lakes. To this end, we introduce LAKEQA, a comprehensive benchmark for search-centric question answering over data lakes that jointly emphasizes *searching* and *reasoning* capabilities. LAKEQA is built on a heterogeneous collection of ~ 9.5 TB text resources from Wikipedia and open-source government data, spanning structured and unstructured data. To ensure the quality of LAKEQA’s tasks, each sample is annotated by at least one Ph.D level expert. Each task requires long-horizon multi-hop reasoning with implicit intermediate steps: agents need to discover the correct document(s) and then compose evidence across sources to produce the answer. Intensive experiment results on seven frontier LLMs have demonstrated that LAKEQA is challenging. For instance, GPT-5.2 only obtains an exact matching score of 14.73% on LAKEQA. Overall LAKEQA provides a realistic testbed for developing LLM agents that can both *find* and *analyze* data in modern data lakes.

1. Introduction

Recent large language models (LLMs) have shown rapid progress on Question Answering (QA) tasks such as reading comprehension (Rajpurkar et al., 2018; Kwiatkowski et al., 2019; Khashabi et al., 2018; Dua et al., 2019) and multihop reasoning (Yang et al., 2018; Talmor & Berant, 2018; Trivedi et al., 2022; Wu et al., 2025; Chen et al., 2020b). Yet, the evidence needed to solve each task in these datasets are largely provided – either already selected by the dataset creator or easily retrievable within a narrow search space.

However, more and more attention has been paid to QA over

an arbitrary data source, i.e., enterprise data warehouses, where users begin with only a question and the supporting evidence is distributed across a massive data lake in multiple documents rather than provided within the LLM’s context window. As a result, an agent trying to answer a natural-language question *must* undergo many rounds of search and reasoning because perfect indices and retrieval techniques do not currently exist. We formalize this capability as **Exploratory Question Answering (EQA)**, where an LLM-powered agent repeatedly alternates between (i) reasoning about the missing evidence required to answer the question and (ii) searching for documents that contain that evidence.

Unfortunately, existing benchmarks (Yang et al., 2018; Talmor & Berant, 2018; Geva et al., 2021; Chen et al., 2020b;a; Trivedi et al., 2022) are insufficient for evaluating EQA because their tasks only require a few documents selected from a small, curated corpus with limited search or limited reasoning iterations. For instance, OTT-QA emphasizes document retrieval but exhibits shallow reasoning, with fewer than two documents needed on average to answer a question. In contrast, MMQA (Wu et al., 2025) increases reasoning complexity, but does so by substantially constraining the search space to only thousands of documents.

To address this limitation, we propose LAKEQA, a benchmark for EQA over a data lake of ~ 9.5 TB spanning ~ 40 million of heterogeneous documents—the first benchmark performed at this scale. The collection includes structured data (csv, json) and unstructured data (txt, pdf, html), with individual tabular data containing millions of rows and spanning gigabytes. Our benchmark introduces three major challenges that mirror ad-hoc analytical workflows in modern data lakes and enterprise warehouses

(1) Missing evidence and high search density. The agent must identify which documents contain the required evidence from a vast collection, where even enumerating document names is prohibitively expensive. In LAKEQA, answering a question may require locating a single relevant document among over one million candidates, as reflected by the Search Density column in Table 1, demanding effective data discovery strategies.

(2) Abundance and heterogeneity. The collection spans many documents of various data types and sizes—ranging from kilobyte text files to gigabyte-scale tables with mil-

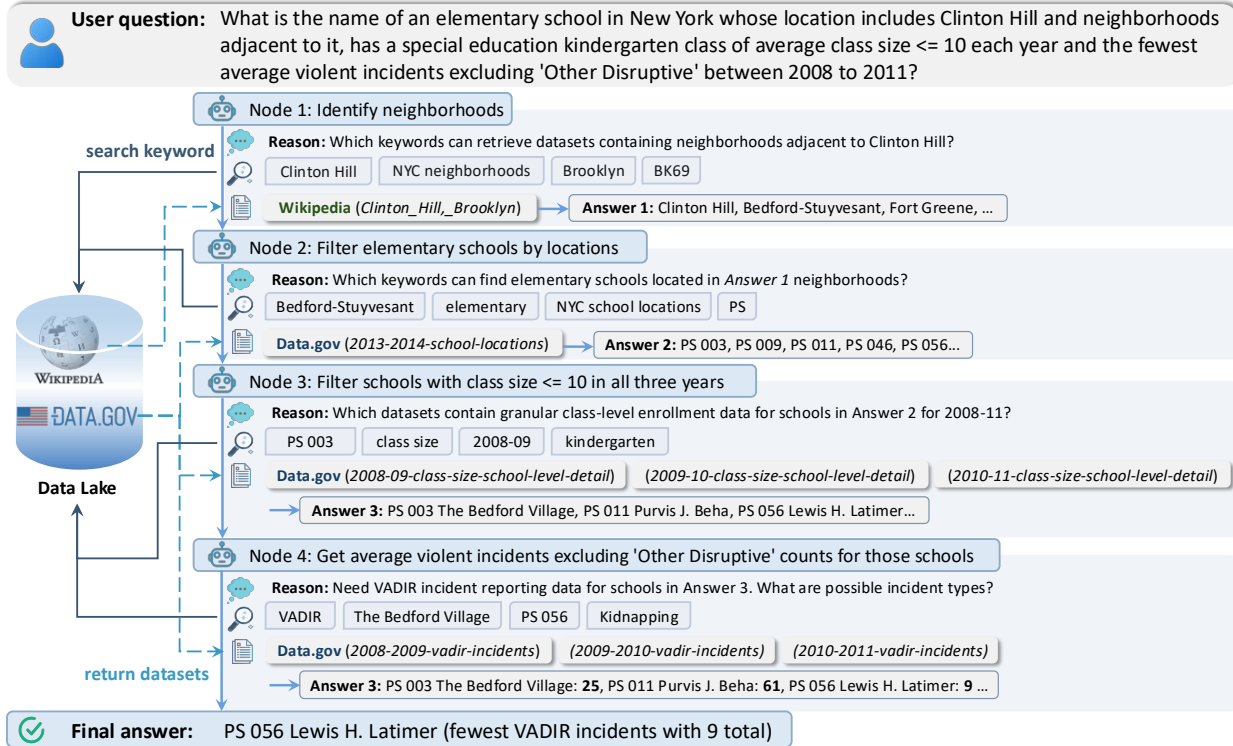


Figure 1. An illustrative task in LAKEQA demonstrating multi-hop exploratory question answering over a heterogeneous data lake. Given a complex natural language user question, an agent must iteratively decompose it into a sequence of sub-tasks that each depends on the answers from previous sub-tasks.

lions of rows—beyond the curated contexts of reading-comprehension benchmarks (Yang et al., 2018) or table-centric QA benchmarks (Wu et al., 2025). Moreover, unlike Wikipedia-based benchmarks where tables and their metadata appear together in a single webpage, LAKEQA separates them into different documents, requiring agents to locate both the data and its metadata and reason over their combination.

(3) Exploratory planning and high reasoning density.

Due to the sheer volume of the data lake, content-level keyword search over all documents is computationally infeasible. An agent must repeatedly reason on what missing information is needed and propose targeted search queries with strong filtering keywords, inspect results, refine search terms and issue successive queries to gather and verify evidence before synthesizing a final answer. Specifically, tasks in LAKEQA require an average of 7.67 documents (Reasoning Density in Table 1), more than double that of existing benchmarks’ documents per task. This high reasoning density induces long-horizon exploratory reasoning chains.

Example 1.1. Figure 1 illustrates an example LAKEQA task. The user question is decomposed into a sequence of subtasks (Nodes 1-4), where each node depends on the output of its predecessor. For example, the agent must first identify relevant neighborhoods (Node 1) before identifying

schools to filter on (Node 2). It must find school that satisfy class size constraints (Node 3) before retrieving incident data (Node 4). At each reasoning step, the agent formulates search queries (blue boxes), retrieves candidate documents from Wikipedia or Data.gov (dashed arrows), extracts intermediate answers used in subsequent steps, and composes the results to synthesize a final answer.

This chained dependency structure increases both reasoning and search difficulty: errors or incomplete retrievals at earlier nodes propagate downstream, and the search space at each step is conditioned on prior results rather than being known upfront.

In addition to curating a large data lake and constructing the benchmark, we also ensure that LAKEQA mitigates the training-data contamination by using public government documents with *analytical statistics* rather than factual Wikipedia paragraphs that may already appear in an LLM’s pretraining data. To ensure high benchmark quality, all 1005 tasks in our benchmark are reviewed by at least three annotators, including at least one Ph.D. student in computer science with data-related research experience. Given the complexity of LAKEQA, we release two benchmark variants: LAKEQA-full and LAKEQA-mini. LAKEQA-mini is a 135-task stratified subset of LAKEQA-full that preserves

the distribution of reasoning density, enabling rapid iteration and quick evaluation on EQA.

Finally, we conduct extensive experiments with seven state-of-the-art LLMs spanning both proprietary and open-source model families. Each model is equipped with a tool suite that mirrors realistic data analysis workflows over a data lake. Despite access to search and inspection tools, all models achieve $< 30\%$ end-to-end accuracy over LAKEQA-full, indicating that EQA remains challenging even for frontier LLMs. By analyzing agent traces, we found that failures are dominated by *missing evidence*: models frequently fail to discover or open the gold documents needed to answer the question. Moreover, accuracy degrades sharply as reasoning density increases, consistent with long-horizon exploration amplifying error accumulation. Overall, these results point to effective search and exploration as the primary bottleneck for current LLM agents on EQA.

2. Related Work

In this section, we survey recent benchmarks and evaluation settings for question answering. We first review multi-hop question-answering over text, then consider QA over heterogeneous data sources. Finally, we discuss open-domain benchmarks for data discovery. Table 1 summarizes existing question-answering benchmarks and highlights how LAKEQA uniquely combines search density, reasoning density, and realistic data lake scale.

Multi-Hop QA over Text. Early benchmarks established compositional reasoning paradigms over text corpora, with limited retrieval scope. COMPLEXWEBQUESTION (Talmor & Berant, 2018) decomposes questions into symbolic operations over knowledge bases (KB), though it remained limited to KB-anchored entities, but provides gold supporting paragraphs and does not require search. HotpotQA (Yang et al., 2018) advances explicit multi-hop reasoning over Wikipedia by requiring hyperlinks across multiple documents, and MuSiQue (Trivedi et al., 2022) increased reasoning density but provides gold supporting paragraphs and does not require search. STRATEGYQA (Geva et al., 2021) shifts toward *implicit multi-hop reasoning*, where intermediate steps must be discovered, increasing reasoning density within Wikipedia corpora. In contrast, LAKEQA significantly scales both reasoning depth and retrieval complexity as shown in Table 1.

Question Answering over Heterogeneous Data. Recent benchmarks extend QA beyond text to heterogeneous modalities. FETAQA (Nan et al., 2022) and MMQA (Wu et al., 2025) focus on intra-table or multi-table relational reasoning for tabular QA, but do not require large-scale retrieval. HYBRIDQA (Chen et al., 2020b) combines Wikipedia tables with linked passages for table-text reasoning while pro-

viding all relevant context. OTT-QA (Chen et al., 2020a) decontextualizes HYBRIDQA’s questions and introduces open-domain retrieval over a large corpus of passages and table segments, while retaining relatively shallow reasoning depth. MultiModalQA (Talmor et al., 2021) also incorporates images, but structures questions with pre-associated context, thus reducing search density. Unlike these benchmarks that provide relevant context or operate over curated collections, LAKEQA requires agents to discover evidence from a ~ 40 million document collection spanning structured and unstructured data across more diverse sources and domains as shown in Figure 3.

Open-Domain Data Discovery. Recent benchmarks have emphasized search density for the problem of multi-hop question answering. BROWSECOMP (Wei et al., 2025) moves beyond static corpora to the open web, thus requiring iterative retrieval. MM-BROWSECOMP (Li et al., 2025) extends BROWSECOMP to multimodal open-domain reasoning across text and images. While BROWSECOMP and MM-BROWSECOMP focus on search in an open web environment, LAKEQA targets data discovery at a data-lake scale, where agents must reason over structured and unstructured data.

The data management community has conducted extensive work on document search over data lakes, focusing on retrieving relevant documents for keyword or tabular queries (Fan et al., 2023; Chapman et al., 2020; Zhang & Balog, 2020; Brickley et al., 2019; Castelo et al., 2021). Benchmarks for document retrieval over government open data to answer keyword queries (Kato et al., 2021; Chen et al., 2020c; Zhang et al., 2025) address the scale and heterogeneity of data lakes (Nargesian et al., 2019). However, they do not consider multi-hop question-answering as a downstream task. In contrast, LAKEQA treats data retrieval as a necessary component of multi-hop question answering, combining high search density with high reasoning density.

3. Benchmark Design

Our design is guided by the following principles. *Diversity*: real-world data lakes contain messy, heterogeneous data, instead of a homogeneous corpus containing only paragraphs or tables. As a result, our collection intentionally mixes structured and unstructured formats. *Reasoning*: existing QA benchmarks often provide pre-selected evidence; we design tasks that require long-horizon multi-hop reasoning where intermediate steps are implicit and must be discovered. *Search*: many benchmarks operate over small corpora that allow brute-force retrieval; we construct a large-scale data lake where agents must reason about what information they need before they can find it.

Table 1. Comparison of question-answering benchmarks. LAKEQA uniquely combines high search density, high reasoning density, and operates over a realistic large-scale data lake. Text refers to unstructured text. Rea. = Reasoning, Srch. = Search, Dens. = Density, Topo. = Topology.

Benchmark	Modalities	Reas. Dens.	Reas. Topo.	Srch. Dens.	Srch. Space
ComplexWebQuestion (Talmor & Berant, 2018)	Text	2.90	Graph	–	KB (Freebase)
HotpotQA (Yang et al., 2018)	Text	2.04	Graph	4.92	5M Wikipedia passages
MuSiQue (Trivedi et al., 2022)	Text	2.49	Graph	8.54	–
StrategyQA (Geva et al., 2021)	Text	2.94	Chain	–	Wikipedia corpus
BrowseComp (Wei et al., 2025)	Text	–	–	–	Open Web
FeTaQA (Nan et al., 2022)	Table	3.58	Chain	1	–
MMQA (Wu et al., 2025)	Table	2.58	Chain	1064.4	Curated table collection
HybridQA (Chen et al., 2020b)	Text + Table	1.76	Chain	21.26	–
OTT-QA (Chen et al., 2020a)	Text + Table	1.77	Chain	> 1M	5M passages + 5M table segments
MultiModalQA (Talmor et al., 2021)	Text + Table + Image	1.635	Chain	13.1	–
MM-BrowseComp (Li et al., 2025)	Text + Image	–	–	–	Open Web
LAKEQA	Text + Table	7.67	Graph	>1M	~40 million documents

3.1. Task Definition

Each task in LAKEQA consists of a natural-language question Q and the data lake $\mathcal{D}$. Here, $\mathcal{D} = \{D_1, \dots, D_n\}$ where each element D_i represents an individual document from Data.gov or Wikipedia. The goal is to let the agent explore $\mathcal{D}$ to (i) identify a set of relevant documents $\{D_1, \dots, D_k\} \subseteq \mathcal{D}$, (ii) extract supporting facts and/or compute aggregated statistics from the retrieved files, and (iii) synthesize a final answer A that answers Q , potentially by comparing evidence across multiple documents. An agent is provided with a fixed set of tools $\mathcal{T}$ for discovery, retrieval, and local analysis. Table 2 summarizes the available tools. Due to the scale of $\mathcal{D}$ ($\approx 40M$ documents), exhaustive enumeration or content-level search is computationally infeasible. Thus, agents must strategically use the search tool to prune the search space using ontology tags before examining document contents.

3.2. Data Lake Construction

We collect data from open data repositories including Wikipedia and data.gov. We use the latest English Wikipedia dump¹ with a compressed size of 24.05 GB, and Harvard LIL’s data.gov archive² with 17.9 TB in size. Files are hosted in Amazon S3 and organized by a unique *dataset ID*, which we define as the canonical identifier used as the S3 prefix. For Wikipedia, the dataset id is the page identifier (title-derived), and each S3 directory contains two files: `content.txt` with the page’s wikitext content and `metadata.json` with page metadata. For data.gov, the dataset id is the data.gov webpage identifier (catalog record), and each S3 directory contains all harvested artifacts for that record, including metadata (`json`), tabular resources (e.g., `csv` and `json`), provider webpages (`html`), and linked reports (`pdf` and `txt`).

¹<https://dumps.wikimedia.org/enwiki/latest/enwiki-latest-pages-articles.xml.bz2>

²<https://source.coop/harvard-lil/gov-data>

To ensure stability and reduce redundancy, we deduplicate Data.gov resources by retaining a single canonical copy when multiple representations of the same resource (e.g., `csv` and `json`) exist for the same resource. This reduces the total stored size to ≈ 9.5 TB and yields $\approx 40M$ unique files. Additional details of the collection and deduplication pipeline are provided in Section C.

3.3. Task Creation Process

Inspired by the COMPOSITION operation in (Talmor & Berant, 2018), annotators create long-horizon multi-hop questions spanning over multiple data sources in LAKEQA through an iterative process illustrated in Figure 2 (2).

Subquestion Construction. To begin with, an annotator starts with an arbitrary document and derives a *fact* from that document. A *fact* is of the shape $\mathcal{F} : \mathcal{E} \Leftrightarrow \mathcal{C}$ where $\mathcal{E}$ is a set of entities, and $\mathcal{C}$ is a sufficient and necessary condition of $\mathcal{E}$. Here, the symbol $\Leftrightarrow$ denotes a bidirectional (if-and-only-if) relationship between entities and the condition. For example, in Figure 2 (2), “Barack Obama is the 44th president of the United States” is a fact where $\mathcal{E} = \{\text{Barack Obama}\}$ and $\mathcal{C} = \text{“44th president of the United States”}$. Each subquestion is a pair of question and answer $(Q, \mathcal{E})$ where Q has the form “What is $\mathcal{C}$?” and is created by reverse engineering a *fact*, i.e., by converting the condition $\mathcal{C}$ into a natural-language question. The *if and only if* requirement ensures that the correct answer to Q is exactly $\mathcal{E}$.

Subquestion Chaining. Starting from a subquestion $(Q_0, \mathcal{E}_0)$, annotators search for a dataset in the data lake containing a new fact $\mathcal{E}_1 \Leftrightarrow \mathcal{C}_1 \wedge \mathcal{E}_0$, where $\wedge$ denotes logical conjunction, meaning that $\mathcal{E}_1$ is characterized by a condition $\mathcal{C}_1$ that additionally depends on the entity set $\mathcal{E}_0$. This yields the next subquestion $(Q_1, \mathcal{E}_1)$, where Q_1 asks for the entity set satisfying $\mathcal{C}_1 \wedge \mathcal{E}_0$. In other words, correctly answering Q_0 is necessary for answering Q_1 . For example, in Figure 2 (2), Q_0 is “Who is the 44th president of the United States?”

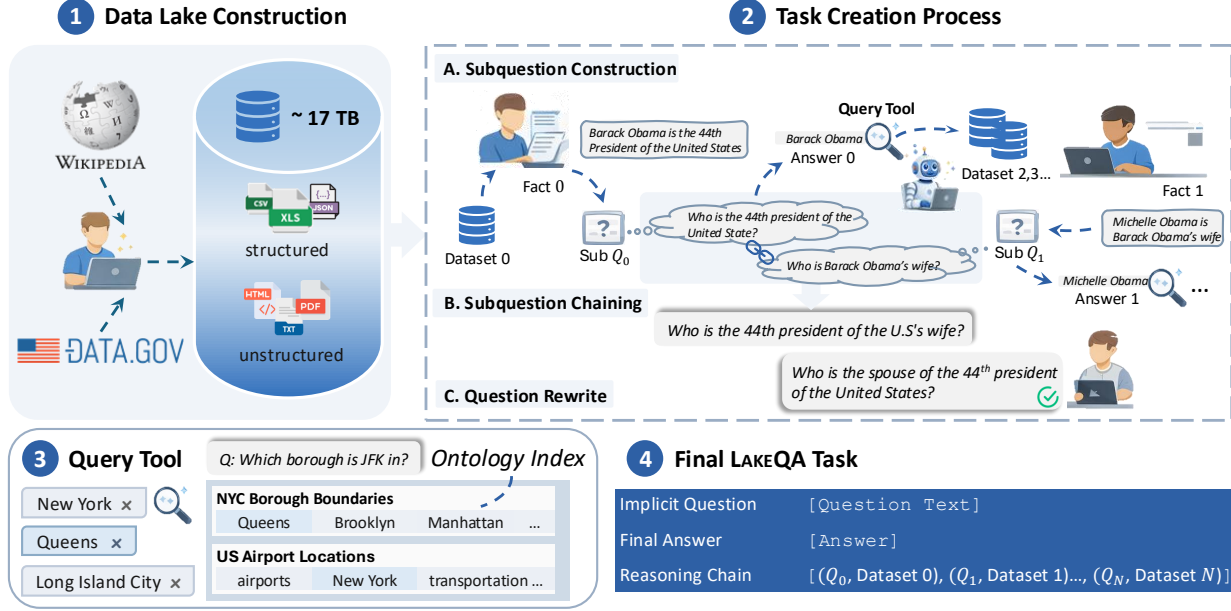


Figure 2. Overview of LAKEQA creation process. (1) *Data Lake Construction*: We construct a heterogeneous collection of ~ 17 TB from Wikipedia and data.gov, including both structured and unstructured data. (2) *Task Creation*: Annotators create multi-hop QA tasks through an iterative annotation process: we first derive facts from document and reverse-engineer them into subquestions (A), then chain subquestions by finding new facts whose conditions depend on previous answers (B), and finally rewrite the composed question to avoid leaking document identifiers (C). (3) *Query Tool*: Throughout annotation, an ontology-indexed search interface helps annotators efficiently search the large collection. (4) *Final Task Format*: Each task in LAKEQA comprises an implicit question (with reasoning steps hidden), a ground-truth answer, and the reasoning chain documenting the evidence trail.

Table 2. Tool interface available to agents in LAKEQA.

Tool	Input	Output / Purpose
<code>search(query)</code>	keyword or tag string	Returns a ranked list of dataset IDs relevant to the query.
<code>listdata(dataset-ids)</code>	list of dataset IDs	Lists the files available under each dataset ID directory.
<code>download(dataset-ids)</code>	list of dataset IDs	Downloads all files for each dataset ID into a per-task local sandbox.
<code>inspect(path)</code>	file path (e.g. <code><id>/<file></code>)	Returns the first k characters (a lightweight preview).
<code>query(path, q)</code>	file path and query q	Queries over locally downloaded data (e.g., filter, aggregate).

with answer $\mathcal{E}_0 = \{\text{Barack Obama}\}$. A dataset may contain the fact “Michelle Obama is Barack Obama’s wife”, whose condition (“wife of Barack Obama”) explicitly depends on the entity identified in the previous step ($\mathcal{E}_0$). This induces the next subquestion Q_1 : “Who is Barack Obama’s wife?” Composing subquestions Q_1 with Q_0 gives the multi-hop question “Who is the 44th president of the United States’ wife?” with answer $\mathcal{E}_1 = \{\text{Michelle Obama}\}$.

Question Rewrite. To emphasize the role of search, we rewrite questions to avoid explicitly mentioning document identifiers—for example, the term `vadir-incidents` in Figure 1. This string is an internal shorthand for “violent and disruptive incidents,” and including it in the final question would leak an obvious search key, making the agent’s exploration substantially easier. Accordingly, annotators are instructed to consult the metadata of each document in-

involved in a task and rewrite the final question using natural language descriptions of the relevant concepts rather than document-specific identifiers.

Quality Control. Recent work has identified upwards of 66% annotation error rates in text-to-SQL benchmarks (Jin et al., 2026b;a), which distorts the reported accuracies and leaderboard rankings. To avoid such annotation errors, each task in LAKEQA undergoes multi-stage quality checks by at least three independent annotators where at least one of the annotators is a computer science Ph.D. More specifically, at least one annotator is responsible to verify the *iff* requirement in $\mathcal{C} \Leftrightarrow \mathcal{E}$ to ensure that the reverse-engineered subquestion “What is $\mathcal{C}$?” returns *exactly* $\mathcal{E}$ (no missing or extraneous entities) under the document’s execution semantics. The Ph.D. annotator is responsible for checking that (a) the composed multi-hop question is unambiguous in natural

Table 3. LAKEQA statistics.

Statistics	LAKEQA
Average Wikipedia dataset IDs per task	1.98
Average Data.gov dataset IDs per task	5.23
Unique Wikipedia dataset IDs	376
Unique Data.gov dataset IDs	555
Maximum Question Length	390
Average Question Length	110.5

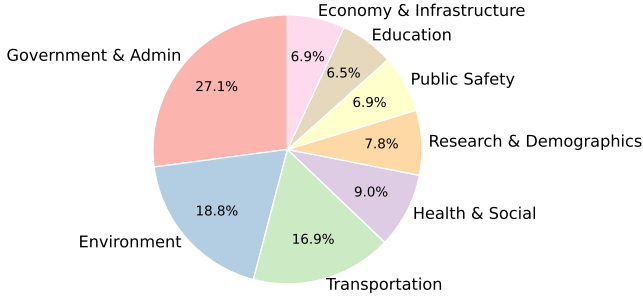


Figure 3. Distribution of task domains.

language, (b) the final answer is uniquely determined by the referenced evidence, and (c) the final question does not leak document identifiers or trivial search keys. Tasks that fail any check are either revised with corrected evidence trails or removed from the benchmark. Detailed annotation guideline can be found in

3.4. Benchmark Statistics

We summarize key statistics of LAKEQA in Table 3. We measure question length in tokens by tokenizing with the `cl100k-base` encoder. We derive each task’s domain labels from the topic metadata provided by Data.gov. Because a task may span multiple domains, we treat domain assignment as multi-label and count each task toward every associated domain. This yields the domain distribution shown in Section D.

4. Experiment

We design experiments based on the LAKEQA benchmark to answer the following major questions: **RQ1**: How do different LLMs perform on EQA tasks? **RQ2**: What is the main reason for LLMs’ failure? **RQ3**: How does LLMs’ performance change with regard to the search and reasoning density?

4.1. Experimental Settings

Datasets. We divide our LAKEQA benchmark into two parts, LAKEQA-mini (135 tasks) and LAKEQA-full (1005 tasks) LAKEQA-mini is a stratified subset of LAKEQA-full that preserves the distribution of reasoning density, enabling rapid iteration and quick evaluation on EQA. Due to the complexity of LAKEQA, LAKEQA-mini is intended for fast development validation, or for those with limited computing resources.

Models. We evaluate our benchmark on seven frontier LLMs spanning both proprietary and open-source model families. For proprietary models, we include OpenAI `gpt-5.2` and `gpt-5-mini`, and Anthropic Claude Haiku 4.5, Claude Sonnet 4.5, and Claude Opus 4.5. For open-sourced models, we include DeepSeek-R1 and Llama-3.3-70B-Instruct. We only evaluate Claude Sonnet 4.5 on LAKEQA-mini due to its high cost.

Evaluation Metrics. We report end-to-end task performance using *Exact Match* (EM) against the ground-truth answer provided and verified by multiple annotators, together with the *wall-clock runtime* and *dollar cost* per task (computed from provider pricing and the model’s measured token usage / API billing units). To study the role of search in the evaluation, we additionally evaluate *data discovery* via dataset-level precision, recall, and F1 under two dataset collections induced by the agent’s traces: first, the *retrieval set* $\mathcal{D}_{\text{ret}}$ contains the union of all dataset-IDs agents had access to through discovery, regardless of whether it was subsequently used. Second, the *accessed set* $\mathcal{D}_{\text{acc}}$ contains dataset-IDs the agent actually *opened/queried* towards constructing its answer. Then, we compute precision and recall by comparing $\mathcal{D}_{\text{ret}}$ and $\mathcal{D}_{\text{acc}}$ to the annotated golden set $\mathcal{D}^*$ for each task. The gap between $\mathcal{D}_{\text{ret}}$ and $\mathcal{D}_{\text{acc}}$ can be interpreted as *reasoning failure*, indicating that the agent mistakenly discards documents relevant to the final question. By contrast, low recall of $\mathcal{D}_{\text{ret}}$ can be interpreted as *searching failure*, indicating that an agent was not even able to pinpoint what to query to find the relevant documents.

Overall Flow. For each task in LAKEQA, an agent is presented with a natural language question and a set of tools described in Table 2, the agent iteratively selects exactly one tool call each round, observes the tool result, and updates its internal state using the full tool-call history. The loop terminates when the agent submits an answer, reaches a turn limit, or timeout. We record the final answer reported by an agent to conduct the evaluation. Further details, including agent prompts and tool signatures, are provided in Section F.

Table 4. Results on the LAKEQA-full.

Dataset	LAKEQA-full								
Evaluation Methods	End-to-end			Accessed Set $\mathcal{D}_{acc}$			Retrieval Set $\mathcal{D}_{ret}$		
Metrics	EM(%) $\uparrow$	Runtime(s) $\downarrow$	Cost(\$) $\downarrow$	P(%) $\uparrow$	R(%) $\uparrow$	F1(%) $\uparrow$	P(%) $\uparrow$	R(%) $\uparrow$	F1(%) $\uparrow$
<i>Open Source LLMs</i>									
Llama-3.3-70B	6.57	25.69	0.02	5.24	2.68	26.73	1.02	5.41	6.24
DeepSeek-R1	20.60	92.00	0.11	33.91	11.01	30.72	3.11	15.40	7.59
<i>Proprietary LLMs</i>									
Claude-haiku-4.5	11.94	70.51	0.31	29.23	25.31	36.88	2.24	34.67	5.00
Claude-sonnet-4.5	23.08	124.45	0.96	34.18	33.27	40.03	2.61	42.12	5.86
GPT-5-mini	10.95	197.75	0.08	32.18	26.91	34.20	1.56	35.34	3.40
GPT-5.2	14.73	66.49	0.39	35.89	25.68	38.93	2.45	30.33	5.79

Table 5. Results on the LAKEQA-mini.

Dataset	LAKEQA-mini								
Evaluation Methods	End-to-end			Accessed Set $\mathcal{D}_{acc}$			Retrieval Set $\mathcal{D}_{ret}$		
Metrics	EM(%) $\uparrow$	Runtime(s) $\downarrow$	Cost(\$) $\downarrow$	P(%) $\uparrow$	R(%) $\uparrow$	F1(%) $\uparrow$	P(%) $\uparrow$	R(%) $\uparrow$	F1(%) $\uparrow$
<i>Open Source LLMs</i>									
Llama-3.3-70B	8.89	27.67	0.03	3.73	1.86	23.59	0.76	5.67	6.37
DeepSeek-R1	23.70	83.62	0.10	23.01	7.88	25.68	2.51	10.46	7.01
<i>Proprietary LLMs</i>									
Claude-haiku-4.5	12.59	67.77	0.30	26.03	21.66	38.96	1.96	28.87	5.08
Claude-opus-4.5	37.04	113.28	1.55	45.38	41.14	50.82	4.61	51.88	9.20
Claude-sonnet-4.5	25.93	129.69	0.92	36.80	32.56	47.61	2.84	44.27	6.86
GPT-5-mini	11.85	192.32	0.08	35.69	24.88	38.95	1.54	32.82	3.73
GPT-5.2	14.81	67.14	0.39	32.48	22.81	40.88	2.59	31.34	6.40

4.2. Results & Analysis

In this section, we discuss and analyze our experiment results reported in Tables 4 and 5.

RQ1: How do different LLMs perform on EQA tasks. Overall, all models achieve relatively low EM, indicating that EQA over a large, heterogeneous data lake remains challenging even for frontier LLMs. On LAKEQA-full, the best end-to-end performance is achieved by Anthropic Claude-sonnet-4.5 (23.08% EM), followed by DeepSeek-R1 (20.60%) and GPT-5.2 (14.73%). On LAKEQA-mini, the strongest model is Claude-opus-4.5 (37.04% EM), with Claude-sonnet-4.5 (25.93%) and DeepSeek-R1 (23.70%) next, suggesting that increased capability improves EQA but is still far from solving the problem.

Finally, efficiency differs dramatically across models. For example, GPT-5-mini is inexpensive per task but has the longest runtime, whereas GPT-5.2 and Claude-haiku-4.5 are faster but more costly, and

Claude-opus-4.5 delivers the best accuracy on LAKEQA-mini at substantially higher cost. This is because GPT-5-mini often write inefficient code to query large datasets where larger models tend to avoid.

Takeaway: EQA presents a three-way trade-off between accuracy, runtime, and dollar cost for LLMs; however, the current Pareto front for the trade-off remains low.

RQ2: What is the main reason for LLMs’ failure. Across all models, the main cause of failure comes from not being able to find the gold datasets required to answer the question. Specifically, in both LAKEQA-full and LAKEQA-mini, the recall of $\mathcal{D}_{acc}$ and $\mathcal{D}_{ret}$ are low, where the recall of $\mathcal{D}_{ret}$ is only slightly higher. This gap suggests the bottleneck occurs early: models often fail to even surface the relevant datasets during search.

Moreover, even though GPT-5.2 achieves higher precision on $\mathcal{D}_{acc}$ than Claude-sonnet-4.5, yet its end-to-end EM on LAKEQA-full is significantly lower. This indicates that conservative exploration-retrieving and open-

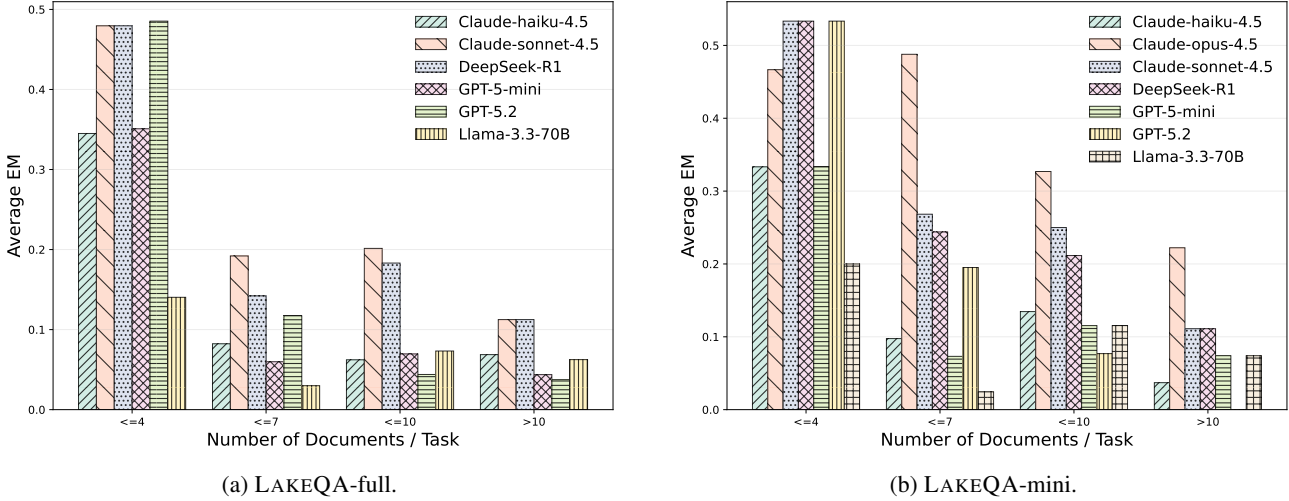


Figure 4. Average EM on LAKEQA stratified by documents per task; each bar reports the mean EM over tasks in the corresponding range.

ing only a small number of highly “confident” datasets—can yield high precision but low coverage, which is fatal in EQA. In contrast, effective agents must balance precision with recall by exploring broadly enough to capture the necessary evidence, even at the cost of inspecting irrelevant ones.

Takeaway: search and exploration (document discovery) is the primary bottleneck; reasoning over found evidence is secondary.

RQ3: How does LLMs performance change with regard to the search and reasoning density. Figure 4 shows a clear and consistent trend across all evaluated models: *as the number of documents required per task increases, average EM drops substantially*. When tasks require only a small evidence set (e.g., ≤ 4 documents), models achieve their highest EM, suggesting that the search component is often manageable over tasks with small or moderate reasoning density. That is, when the rounds of exploration in the data lake is small, agents are able to reason over what information they are looking for and know what to search to retrieve them. However, as the reasoning density increases where tasks expand to involve more documents (e.g., ≥ 7 and especially > 10), performance downgrade drastically. This regime forces search to be tightly interleaved with multi-step reasoning: the agent must repeatedly retrieve, maintain intermediate hypotheses, and integrate evidence across documents. Each additional document effectively adds another dependent step, amplifying error accumulation from imperfect retrieval, missed constraints, and faulty cross-document aggregation, which makes the overall task dramatically harder and drives EM toward $< 10\%$.

Takeaway: EQA performance degrades steeply with higher search and reasoning density because long-horizon exploration and multi-document integration magnify compound-

ing errors.

5. Conclusion

In conclusion, we introduced LAKEQA, a benchmark built up on a million-scale data lake for Exploratory Question Answering (EQA). LAKEQA is designed to examine LLM agents in realistic settings where evidence is not provided and must be discovered through iterative search and reasoning. By constructing LAKEQA over public government datasets with analytical statistics and employing dedicated task creation process with multiple annotators, we aim to reduce contamination and ensure high-quality evaluation. Our experiments with seven frontier LLMs show that EQA remains far from solved: even with access to search and inspection tools, end-to-end accuracy remains low, and performance drops sharply as tasks require longer reasoning chains and more evidence sources. Trace-based analysis further reveals that the main bottleneck is document discovery—models frequently fail to retrieve and open the gold documents—while conservative exploration strategies that yield high precision but low recall are heavily penalized.

Impact Statements

This paper presents a benchmark designed to advance research in machine learning, particularly in question answering, retrieval, and reasoning over large-scale data lakes. The benchmark is constructed entirely from publicly available data and is intended as an evaluation resource rather than a deployed system. We do not foresee societal or ethical impacts beyond those commonly associated with progress in data-driven machine learning methods.

References

- Brickley, D., Burgess, M., and Noy, N. F. Google Dataset Search: Building a search engine for datasets in an open Web ecosystem. In *WWW*, pp. 1365–1375, 2019.
- Castelo, S., Rampin, R., Santos, A. S. R., Bessa, A., Chirigati, F., and Freire, J. Auctus: A Dataset Search Engine for Data Discovery and Augmentation. *Proc. VLDB Endow.*, 14(12):2791–2794, 2021.
- Chapman, A., Simperl, E., Koesten, L., Konstantinidis, G., Ibáñez, L.-D., Kacprzak, E., and Groth, P. Dataset search: a survey. *The VLDB Journal*, 29(1):251–272, 2020.
- Chen, W., Chang, M.-W., Schlinger, E., Wang, W., and Cohen, W. W. Open question answering over tables and text. *arXiv preprint arXiv:2010.10439*, 2020a.
- Chen, W., Zha, H., Chen, Z., Xiong, W., Wang, H., and Wang, W. Y. Hybridqa: A dataset of multi-hop question answering over tabular and textual data. In *Findings of the Association for Computational Linguistics: EMNLP 2020*, pp. 1026–1036, 2020b.
- Chen, Z., Jia, H., Heflin, J., and Davison, B. D. Leveraging schema labels to enhance dataset search. In *European Conference on Information Retrieval*, pp. 267–280. Springer, 2020c.
- Clark, C., Lee, K., Chang, M.-W., Kwiatkowski, T., Collins, M., and Toutanova, K. Boolq: Exploring the surprising difficulty of natural yes/no questions. *arXiv preprint arXiv:1905.10044*, 2019.
- Dua, D., Wang, Y., Dasigi, P., Stanovsky, G., Singh, S., and Gardner, M. Drop: A reading comprehension benchmark requiring discrete reasoning over paragraphs. *arXiv preprint arXiv:1903.00161*, 2019.
- Fan, G., Wang, J., Li, Y., and Miller, R. J. Table Discovery in Data Lakes: State-of-the-art and Future Directions. In *SIGMOD Conference Companion*, pp. 69–75. ACM, 2023.
- Geva, M., Khashabi, D., Segal, E., Khot, T., Roth, D., and Berant, J. Did aristotle use a laptop? a question answering benchmark with implicit reasoning strategies. *Transactions of the Association for Computational Linguistics*, 9: 346–361, 2021.
- Jin, T., Choi, Y., Zhu, Y., and Kang, D. Pervasive annotation errors break text-to-sql benchmarks and leaderboards. In *ArXiv*, 2026a. URL <https://api.semanticscholar.org/CorpusID:284704474>.
- Jin, T., Choi, Y., Zhu, Y., and Kang, D. Text-to-sql benchmarks are broken: An in-depth analysis of annotation errors. *CIDR*, 2026b.
- Kato, M. P., Ohshima, H., Liu, Y.-H., and Chen, H.-L. A test collection for ad-hoc dataset retrieval. In *ACM SIGIR*, pp. 2450–2456, 2021.
- Khashabi, D., Chaturvedi, S., Roth, M., Upadhyay, S., and Roth, D. Looking beyond the surface: A challenge set for reading comprehension over multiple sentences. In *Proceedings of the 2018 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies, Volume 1 (Long Papers)*, pp. 252–262, 2018.
- Kwiatkowski, T., Palomaki, J., Redfield, O., Collins, M., Parikh, A., Alberti, C., Epstein, D., Polosukhin, I., Devlin, J., Lee, K., et al. Natural questions: a benchmark for question answering research. *Transactions of the Association for Computational Linguistics*, 7:453–466, 2019.
- Li, S., Bu, X., Wang, W., Liu, J., Dong, J., He, H., Lu, H., Zhang, H., Jing, C., Li, Z., et al. Mm-browsecomp: A comprehensive benchmark for multimodal browsing agents. *arXiv preprint arXiv:2508.13186*, 2025.
- Nan, L., Hsieh, C., Mao, Z., Lin, X. V., Verma, N., Zhang, R., Kryściński, W., Schoelkopf, H., Kong, R., Tang, X., et al. Fetaqa: Free-form table question answering. *Transactions of the Association for Computational Linguistics*, 10:35–49, 2022.
- Nargesian, F., Zhu, E., Miller, R. J., Pu, K. Q., and Arocena, P. C. Data lake management: challenges and opportunities. *Proceedings of the VLDB Endowment*, 12(12): 1986–1989, 2019.
- Rajpurkar, P., Jia, R., and Liang, P. Know what you don’t know: Unanswerable questions for squad. *arXiv preprint arXiv:1806.03822*, 2018.
- Talmor, A. and Berant, J. The web as a knowledge-base for answering complex questions. *arXiv preprint arXiv:1803.06643*, 2018.
- Talmor, A., Yoran, O., Catav, A., Lahav, D., Wang, Y., Asai, A., Ilharco, G., Hajishirzi, H., and Berant, J. Multimodalqa: Complex question answering over text, tables and images. *arXiv preprint arXiv:2104.06039*, 2021.
- Trivedi, H., Balasubramanian, N., Khot, T., and Sabharwal, A. musique: Multihop questions via single-hop question composition. *Transactions of the Association for Computational Linguistics*, 10:539–554, 2022.
- Wei, J., Sun, Z., Papay, S., McKinney, S., Han, J., Fulford, I., Chung, H. W., Passos, A. T., Fedus, W., and Glaese,

- A. Browsecomp: A simple yet challenging benchmark for browsing agents. *arXiv preprint arXiv:2504.12516*, 2025.
- Wu, J., Yang, L., Li, D., Ji, Y., Okumura, M., and Zhang, Y. Mmqa: Evaluating llms with multi-table multi-hop complex questions. In *The Thirteenth International Conference on Learning Representations*, pp. 1, 2025.
- Yang, Z., Qi, P., Zhang, S., Bengio, Y., Cohen, W., Salakhutdinov, R., and Manning, C. D. Hotpotqa: A dataset for diverse, explainable multi-hop question answering. In *Proceedings of the 2018 conference on empirical methods in natural language processing*, pp. 2369–2380, 2018.
- Zhang, H., Liu, Y., Santos, A., Hung, W.-L., and Freire, J. Autoddg: Automated dataset description generation using large language models. *arXiv preprint arXiv:2502.01050*, 2025.
- Zhang, S. and Balog, K. Web table extraction, retrieval, and augmentation: A survey. *ACM Transactions on Intelligent Systems and Technology*, 11(2):1–35, 2020.

A. Task description for Annotators to Characterize Searching and Reasoning Intensity

Let $\mathcal{D}$ be the collection of data source, the goal is to evaluate some statistics about tasks in existing QA benchmarks. For each task with a pair of question and answer (Q, A) , we define the underlying *question decomposition graphs*, or QDGs, as a collection of directed acyclic graphs (DAGs) $\{\mathcal{G}_i\}$. Each DAG $\mathcal{G}_i = (\mathcal{V}, \mathcal{E})$ semantically represents a minimal reasoning trace of deriving the answer A from the question Q . The set of nodes $\mathcal{V}$ encodes questions and the set of directed edges $\mathcal{E}$ encodes dependencies between the questions. Specifically, $\mathcal{V} = \{Q, A\} \cup \{Q_i\}$ where Q is the source node, A is the sink node and $\{Q_i\}$ are sub-questions.

Each sub-question Q_i is an *atomic* question, meaning that Q_i can be answered by *exactly* one piece of data source, in each benchmark, one piece of data source could be a paragraph, a table, or a single image. A directed edge $Q_i \rightarrow Q_j$ exists *if and only if* Q_j depends on the answer of Q_i (guarantees minimality). For example, the question Q : “Who is the spouse of the 44th president of United States?” can be decomposed into a QDG $\mathcal{G}_1$ as $Q \rightarrow Q_1 \rightarrow Q_2 \rightarrow A$ where Q_1 : “Who is the 44th president of United States?” and Q_2 : “Who is the spouse of A_1 ?” where A_1 is the answer of Q_1 . Finally, the final answer A only depends on Q_2 .

In the case where there exists different ways to answer A based on Q , for example, the same Q can be decomposed via a single sub-question Q_3 : “Who is the 44th first lady of United States?”. This induces a second QDG $\mathcal{G}_2$ as $Q \rightarrow Q_3 \rightarrow A$.

Tasks in existing QA benchmarks are constructed by a QDG, and we would like to quantify the reasoning complexity of the tasks. As the first step, we would like to collect the following statistics

- (S1) The smallest number of nodes across all QDGs.
- (S2) The shortest directed path from Q to A over all QDGs.
- (S3) For each QDG, calculate the maximum-cardinality minimal Q - A cut, and return the minimal over all QDGs.

These statistics above are for each task in a benchmark. Over all tasks of the benchmark, calculate the **min**, **max**, **mean** and **median** for each of S1, S2 and S3, and report what are the set of data sources, e.g. a paragraph in Wikipedia, a table in Wikipedia or an image in Wikipedia.

Existing work is either open domain or closed domain, for open domain, each question maps to a search, for close domain, each sub-question maps to a data source. For example, each sub-question in COMPLEXWEBQUESTION maps to an internet search that will return the answer to this sub-question, it is unclear how many data sources each sub-question maps to. On the other hand, each sub-question in HotpotQA maps to a single data source.

B. Extended Related Work

B.1. COMPLEXWEBQUESTION (Talmor & Berant, 2018)

[Text, Open Domain] COMPLEXWEBQUESTIONS targets open-domain questions backed by a search engine. The paper highlights a decomposition framework to parse the input natural language question via a computation tree. The leaf nodes of the computation tree consists of questions that are directly answerable from the search engine; internal nodes apply operators—conjunction (set intersection), comparatives (filters like $>$), superlatives (argmax/argmin) and composition (join)—to combine leaf answers. For example, the leaves “Which films were directed by Christopher Nolan?” and “Which films star Leonardo DiCaprio?” each return a set of films; the parent node applies conjunction to yield the composite question “Which Nolan films star Leonardo DiCaprio?”. In collecting tasks for the benchmark, the authors start from WEBQUESTIONSP questions, where each question is paired with a simple SPARQL query. COMPLEXWEBQUESTIONS generate more complex SPARQL queries by applying one operation in $\{\text{CONJUNCTION}, \text{COMPARATIVE}, \text{SUPERLATIVE}, \text{COMPOSITION}\}$ to a SPARQL query from WEBQUESTIONSP.

- CONJUNCTION takes two SPARQL queries to ensure their intersection is a non-empty set, so the output SPARQL simply concatenates involving conditions from the involving queries.
- COMPARATIVE intakes a SPARQL query and find an attribute where the denotation of the SPARQL query possess in common, and apply a filter on that attribute.

- SUPERLATIVE is mostly similar to COMPARATIVE except that instead of applying a filter, we find an $\text{argmax}/\text{argmin}$.
- COMPOSITION starts with a SPARQL query r , finds an entity e in r , and replace that entity with another question whose answer is $\{e\}$ by looking at the KB and finding unique identifiers of e .

After arriving at the SPARQL for each task, AMT workers are employed to translate them into natural language questions.

B.2. HotpotQA (Yang et al., 2018)

[Text, Retrieval, Explicit Multi-hop Reasoning] HotpotQA is a reading comprehension benchmark that evaluates question answering capabilities of an agent over information retrieval and multi-hop reasoning. Unlike ComplexWebQuestion, whose questions are dependent on the knowledge graph of Wikipedia, which forces the questions to only involve entities from the knowledge graph, which is an incomplete source. HotpotQA proposes a method based on hyperlinks between Wikipedia pages through the following procedures

- The authors first extract the first paragraphs of each Wikipedia page. Then, they treat those paragraphs as nodes and build a hyperlink graph by adding an edge between two nodes if one paragraph contains a hyperlink to another.
- Bridge entities: annotators are presented with a pair of paragraphs linked by an edge in the hyperlink graph in the first step, and propose questions requiring both paragraphs to answer.
- Comparison: the authors curate 42 lists of similar entities from Wikipedia (each list contains many paragraphs) and present two paragraphs from the same list to annotators to create questions like “Who has played more games in NBA, Kobe Bryant or Micheal Jordan?”

Aside from answering questions where the exact context needed are given, HotpotQA allows testing for harder tasks where distractor paragraphs are included in the question to test for robustness and the full Wikipedia paragraphs (5M paragraphs) are included in the question to test for relevant information retrieval.

B.3. MUSEQUE (Trivedi et al., 2022)

[Text, Multi-hop Reasoning, Compositional] MUSIQUE addresses the “shortcut” problem in HOTPOTQA, where models often bypass the intended reasoning chain by relying on single-hop clues or entity overlap. To mitigate this, MUSIQUE introduces a systematic bottom-up construction process. Instead of starting with a complex question and decomposing it, the authors begin with a large pool of 2-hop, 3-hop, and 4-hop reasoning graphs composed of connected single-hop questions from existing datasets (NATURAL QUESTIONS and HOTPOTQA).

B.4. HYBRIDQA (Chen et al., 2020b)

[Text, Table, Explicit Multi-hop Reasoning] HYBRIDQA One key contribution of HYBRIDQA is that it integrates both tables and text into the context of its questions. The data source of HYBRIDQA consists of webtables as tabular datasets and paragraphs as text data from Wikipedia. To create tasks for HYBRIDQA, annotators are given HITs (human intelligence task) where each HIT consists of a single webtable alone with paragraphs linked by hyperlinks in the webtable’s cells³. For each HIT, annotators are tasked to create 6 questions that requires information relying on both tabular and textual information to answer. The questions are created based on the following three atomic reasoning chains:

- Table $\rightarrow$ Passage chain: first uses table-wise operations (equal/greater/less/first/last/ $\text{argmax}/\text{argmin}$) to locate a certain tuple in the table, and retrieves a text span from the passage in the hyperlink of that tuple.
- Passage $\rightarrow$ Table chain: reverse of the first type, first retrieves a paragraph, and asks about a tuple with a hyperlink that points to this paragraph.
- Passage $\rightarrow$ Table $\rightarrow$ Passage: same as the Passage $\rightarrow$ Table chain, but hops back to another Passage (i.e. another hyperlinked cell in the same tuple).

³HYBRIDQA crop at most the first 12 sentences from the introduction paragraph of the Wikipedia page, and small webtables (5-20 rows and 3-6 columns) with hyperlinked cells over 35% of its total cells. This ends up with 13000 webtables.

There are three more types of tasks in HYBRIDQA, which corresponds to the CONJUNCTION, COMPARATIVE and SUPERLATIVE operators in (Talmor & Berant, 2018) to compose more complex questions based on questions created via the three atomic reasoning chains.

B.5. OTT-QA (Chen et al., 2020a)

[Text, Table, Retrieval] OTT-QA builds on top of HYBRIDQA by reusing HYBRIDQA’s questions while requiring an additional retrieval step to search for relevant tables and text. To do so, OTT-QA *decontextualize* tasks in HYBRIDQA by removing context-dependent keywords in the natural language questions. For example, “the players” in a HYBRIDQA question is context dependent because the table in the context is about “Netherlands players”. But this is ambiguous in OTT-QA when relevant context to answer the question requires retrieval. Additionally, the authors of OTT-QA decompose each table into several table segments where each table segment consists of a tuple, the table’s headers, metadata, statistics of the original table, i.e. min/max of a column (table documentations). Together, the candidate pool of retrieval consists of 5M passages and 5M table segments.

B.6. MultiModalQA (Talmor et al., 2021)

[Text, Table, Image, Explicit Multi-hop Reasoning] MultiModalQA extends existing reading comprehension datasets such as Natural Questions (NQ) (Kwiatkowski et al., 2019), BoolQ (Clark et al., 2019), and HotpotQA (Yang et al., 2018) by incorporating multimodal contexts involving texts, tables and images into its questions. To construct the benchmark, annotators first create single-hop, single-modality, and persistent questions—those whose answers are unlikely to change over time. For instance, given an image of the Statue of Liberty, a persistent question might ask what the statue is holding. MultiModalQA adopts text-based questions from existing reading comprehension datasets such as HotpotQA. Finally, more complex tasks are formed by linking single-hop, single-modality questions when their referenced entities coincide. For example, a paragraph stating “Barack Obama was born in Honolulu, United States” and a table listing “Barack Obama as the 44th President of the United States” share the same Wikipedia entity, Barack Obama, and can therefore be linked to form a question requiring information from both the paragraph and the table.

B.7. STRATEGYQA (Geva et al., 2021)

[Text, Open Domain, Implicit Multi-hop Reasoning, Binary Response] STRATEGYQA addresses a key limitation of prior QA benchmarks, where all information needed to answer a question is explicitly stated in the question. Instead, it evaluates implicit multi-hop reasoning: the intermediate steps are not spelled out and must be discovered through exploration and retrieval. To construct the dataset, annotators begin with a seed concept and a target yes/no answer, then craft a strategy question whose solution requires composing several atomic facts, each independently verifiable in Wikipedia but not explicitly mentioned in the question. To ensure feasibility (and avoid ungrounded decompositions), annotators also specify, for every step, a candidate Wikipedia page where the fact can be supported; the released data includes these implicit facts and linked source paragraphs as optional intermediate supervision. The benchmark thus tests open-domain retrieval, composition, and reasoning over implicit evidence—beyond surface cues or single-pass reading.

B.8. FETAQA (Nan et al., 2022)

[Table, Free-form Response, Wikipedia] FETAQA extends existing table QA benchmarks whose answers are typically short answers evaluated by exact matching by introducing long, informative free-form answers grounded in a single Wikipedia table. To construct such questions, FETAQA starts from ToTTo (?), a large-scale table-to-text dataset containing naturally written descriptions fully grounded in Wikipedia tables along with supporting cells highlighted. It then filters ToTTo instances to keep tables of moderate size and descriptions whose highlighted cells span more than a single row or column. Given each instance, annotators are tasked to write a question whose answer is a (possibly slightly edited) description derived from optionally modifying the sentence, table contents, or highlighted region to yield natural question–answer interactions. For automatic evaluation of generated answers, FETAQA reports n-gram overlap metrics (sacreBLEU, ROUGE-1/2/L, METEOR) as well as semantic similarity metrics (BERTScore and BLEURT).

B.9. BROWSECOMP (Wei et al., 2025)

[Open Domain, Retrieval] BROWSECOMP is an open-domain deep research benchmark. It consists of 1,266 extremely challenging “needle-in-a-haystack” questions requiring multi-step web searches to find the answer. In contrast to the difficult

to answer questions, the results are short facts that can be easily verified. The benchmark was constructed by inverting a typical question-creation process: human annotators started with a known fact (the target answer) and then added multiple specific qualifiers or constraints to the query until that fact became the unique solution. To ensure these tasks could not be solved via shortcuts, annotators are tasked to verify that simple search-engine queries (up to 5 attempts) cannot directly reveal the answer and SOTA models (e.g. GPT-4 and OpenAI’s deep research agent) failed to solve each question. Further, if another human could find the answer within 10 minutes, the task was revised with additional criteria to increase its difficulty.

B.10. MM-BROWSECOMP (Li et al., 2025)

[Multimodal, Open Domain, Retrieval] MM-BROWSECOMP extends text-only web browsing benchmarks such as BROWSECOMP to evaluate multimodal web research capabilities. It consists of 224 challenging questions that require agents to retrieve and reason over both textual and visual content encountered during browser-style navigation, where crucial evidence is often embedded in images or videos and cannot be recovered through text-only search. The benchmark follows a similar inverted question-creation process as BROWSECOMP and includes a checklist-based evaluation that verifies whether agents complete essential multimodal reasoning steps, revealing substantial gaps in current multimodal browsing systems.

B.11. MMQA (Wu et al., 2025)

[Multi-Table, Multi-Hop, Tabular QA] MMQA is a multi-table question answering benchmark designed to evaluate retrieval and multi-hop reasoning over interconnected tables. Unlike mMltiModalQA datasets that integrate text and images, this benchmark focuses on reasoning across multiple relational tables, requiring models to identify relevant tables, understand their structural relationships (e.g., key-based joins), and synthesize evidence across hops. The benchmark evaluates performance on table retrieval, relational reasoning, and downstream QA or text-to-SQL tasks, highlighting limitations of current models in complex tabular reasoning settings.

C. Dataset Collection

C.1. Background

This section describes the data sources of LAKEQA, we form a data lake by collecting data from open data repositories including Wikipedia and data.gov. We use the latest English Wikipedia dump⁴ and Harvard LIL’s data.gov archive⁵. Our final goal is to create question-answering tasks (QA-tasks) requiring multi-hop reasoning and multiple data sources (different Wikipedia pages and data sources from data.gov). To do so, we would need to compose information from each data source (i.e. one data source contains information like Law X was modified on date Y and another data source contains information like Study Z was conducted on date Y, thus date Y can be used to connect these two pieces of information together.) An LLM agent will assist on this step with annotators to verify the validity of the created questions.

C.2. Task

Each webpage from Wikipedia and data.gov is considered a data source. Please (1) download the data from the provided links in the footnote; (2) preprocess each data source, place them under a folder that is representative of the data source (i.e. title of the Wikipedia or data.gov webpage) and upload all folders into the S3 bucket you created in the pre-step; and (3) parallelize the data downloading, preprocessing and uploading pipeline to handle the large data source (≈ 20 GB from Wikipedia and ≈ 15 TB from data.gov). Here are (partial) instructions on how to preprocess the data sources, let me know if you encounter any other situation you are uncertain about.

- A data source might contain binary files and pictures that does not contain useful information in creating QA-tasks – remove those files under the folder.
- The raw data source contains html files (and potentially files in other formats) with plenty of redundancy – convert them into txt files by removing redundant characters (i.e. html blocks such as `</text/>` shall be converted into `text`, you can use the `BeautifulSoup` package under the `bs4` library in Python to do so) and extra white spaces (i.e. `/n`, `/t`, etc.), etc.

After you are done, tell me the name of the S3 bucket you created. The deadline is this Saturday. Please don’t hesitate to ping me if you have any question.

D. Distribution of Domains

E. Agent Interface

Tool Implementations. All data access tools operate over a fixed S3 data lake bucket (`lakeqa-yc4103-datalake`) with two namespaces: `wikipedia/` and `datagov/`. Credentials are loaded via environment variables, and all downloads are stored in a per-session sandbox directory.

- `search(prefix)`: Performs an S3 prefix search in both namespaces using `list_objects_v2` with `Prefix` and `Delimiter='/'`. The tool returns dataset identifiers corresponding to directory names under each namespace.
- `search_keyword(keyword)`: Issues external keyword searches to the Wikipedia API and the data.gov `package_search` endpoint to propose candidates, then validates each candidate against S3 by checking for a dataset prefix. Candidates are ranked by a lightweight token overlap score that combines query coverage and token density, and the top results are returned.
- `list_files(dataset_id)`: Lists objects under `<namespace>/<dataset_id>/` using `list_objects_v2` and returns relative file paths and sizes (bounded by a maximum count).
- `download(dataset_id, file_path)`: Downloads a specific object to the local sandbox via `s3.download_file`. The tool creates the required directory structure and returns the local path and file size.
- `inspect_file(dataset_id, file_path)`: Retrieves object metadata with `head_object` and reads the first 64KB using a ranged `get_object` request. It infers a delimiter from the first line (comma, tab, pipe, or semicolon),

⁴<https://dumps.wikimedia.org/enwiki/latest/enwiki-latest-pages-articles.xml.bz2>

⁵<https://source.coop/harvard-lil/gov-data>

Table 6. Mapping of data.gov dataset themes to 8 main categories. Task count indicates the number of benchmark tasks containing datasets from each category.

Main Category	Original Themes	Tasks
Government & Admin	Local Government, Government and Taxes, Administration & Finance, City Government, Finance/Tax/Property, Procurements and Contracts	518
Environment	Geospatial, Energy & Environment, Natural Resources, Locations and Maps, Water, Utilities & Water, Boundaries	360
Transportation	Transportation, Aviation, Roadways and Bridges	324
Health & Social	NCHS, 500 Cities & Places, Health & Human Services, State Drug Utilization, Human Services, Recreation, Disability Compensation, COVID-19	237
Research & Demographics	Research and Statistics, Community Demographics, Demographic, Directory	149
Economy & Infrastructure	Economic Development, Business and Economy, Workforce, City Infrastructure, Sanitation, Built Environment, Agriculture	132
Public Safety	Public Safety, Public Safety and Preparedness	132
Education	Education, Primary & Secondary Ed	124

Table 7. Distribution of benchmark tasks across data.gov theme categories.

Theme Category	Tasks
Government & Admin	518
Environment	360
Transportation	324
Health & Social	237
Research & Demographics	149
Economy & Infrastructure	132
Public Safety	132
Education	124

extracts header columns, and, for JSON-like lines, attempts to parse top-level keys. The tool returns metadata only (no raw content).

- `execute_code(code)`: Executes Python in the sandbox with working directory set to the sandbox root. Common libraries (`pandas`, `json`, `csv`, `os`, `glob`, `re`, `Path`) are preloaded, and two helper variables are provided: `SANDBOX_DIR` and `FILES`. The tool captures `stdout/stderr`, returns errors and tracebacks, and enforces a timeout to prevent inefficient code.
- `get_sandbox_info()`: Enumerates all files in the sandbox via a recursive walk and reports paths and byte sizes.
- `submit_answer(answer, reasoning, sources)`: Terminates the episode and records the final answer and cited sources. The framework expects answers in a normalized bracketed format (e.g., `[123]`).

F. Evaluation Prompt

Listing 1. System prompt and tool schemas used by our data analysis agent.

```

DEFAULT_SYSTEM_PROMPT = """You are a data analysis agent working with PUBLIC GOVERNMENT
DATASETS (data.gov, census, etc.).

## HOW THIS WORKS - READ CAREFULLY
This is an INTERACTIVE system. You output ONE tool call, then STOP. The system executes
your tool and returns the REAL result. Then you are called again to pick the next
tool.

DO NOT:
- Output multiple tool calls
- Simulate or hallucinate results (e.g., {"result": ...} or {"error": ...})
- Continue the conversation yourself
- Make up data

ONLY output a single JSON object with your tool call. The system handles execution.

## DATA ACCESS (each step = one tool call)
- search(prefixes) or search_keyword(keywords) $to$ returns dataset_ids (identifiers,
  not data)
  - Pass a list of strings: search(["climate", "weather"]) or search_keyword(["police",
    "crime"])
- list_files(dataset_ids) $to$ returns file paths in datasets
  - Pass a list of strings: list_files(["dataset1", "dataset2"])
- download(files) $to$ downloads files to sandbox (max 5 per call)
  - Pass a list of {dataset_id, file_path}: download([{"dataset_id": "...", "file_path":
    "..."}])
- execute_code(code) $to$ runs Python on downloaded files (use print(!))
- submit_answer() $to$ when ready to submit final answer

## CRITICAL: VERIFY DATA SOURCES
Dataset names can be misleading! Example: "traffic-incidents-2020" could be from Chicago,
NYC, or any other city.
ALWAYS check metadata before using a dataset:
- Download and read metadata files (e.g., metadata.json, catalog.txt) to find the actual
  source
- Look for: publisher, source, city/state, geographic coverage, agency name
- Verify the data matches what the question asks for (correct city, agency, time period)
- Two datasets with similar names may cover completely different locations!

## TIPS
- use search_keyword for semantic matching, SINGLE word preferred
- Always print() in execute_code to see output
- Check actual column names and date formats in the data
- Use full dataset for final answer, not just samples
- Answer format: [value] only, no labels or units

## TURN AND TIME LIMITS

```

```
- You have LIMITED TURNS. The system will show you remaining turns.
- There is also a TIME LIMIT. Do not waste time on excessive exploration.
- After inspecting files, IMMEDIATELY run execute_code to analyze data.
- If running low on turns or time, prioritize submitting your best answer.""

# Tool schemas for native tool calling (works with both OpenAI and Claude/Bedrock)
TOOL_SCHEMAS = [
    {
        "name": "search",
        "description": "Find datasets by name prefixes. Returns dataset identifiers (not
            data). Search multiple prefixes at once.",
        "parameters": {
            "type": "object",
            "properties": {
                "prefixes": {
                    "type": "array",
                    "items": {"type": "string"},
                    "description": "List of prefixes to search for (e.g., ['austin-police',
                        'climate', 'weather'])"
                }
            },
            "required": ["prefixes"]
        }
    },
    {
        "name": "search_keyword",
        "description": "Semantic keyword search across datasets. Search multiple keywords
            at once. Returns ranked dataset identifiers.",
        "parameters": {
            "type": "object",
            "properties": {
                "keywords": {
                    "type": "array",
                    "items": {"type": "string"},
                    "description": "List of keywords to search for (e.g., ['police', 'crime',
                        'traffic'])"
                },
                "limit": {
                    "type": "integer",
                    "description": "Max results to return (default 20)",
                    "default": 20
                }
            },
            "required": ["keywords"]
        }
    },
    {
        "name": "list_files",
        "description": "List files within datasets. Use dataset_ids from search results.
            List multiple datasets at once.",
        "parameters": {
            "type": "object",
            "properties": {
                "dataset_ids": {
                    "type": "array",
                    "items": {"type": "string"},
                    "description": "List of dataset identifiers from search results (e.g.,
                        ['Barack_Obama', 'climate-data'])"
                }
            },
            "required": ["dataset_ids"]
        }
    }
]
```

```

    "name": "download",
    "description": "Download files from datasets to the local sandbox for analysis. Max
      5 files per call.",
    "parameters": {
      "type": "object",
      "properties": {
        "files": {
          "type": "array",
          "items": {
            "type": "object",
            "properties": {
              "dataset_id": {"type": "string", "description": "Dataset
                identifier"},
              "file_path": {"type": "string", "description": "Path to file within
                the dataset"}
            },
            "required": ["dataset_id", "file_path"]
          },
          "description": "List of files to download (max 5). Each with dataset_id
            and file_path.",
          "maxItems": 5
        }
      },
      "required": ["files"]
    },
  },
  {
    "name": "inspect_file",
    "description": "Inspect a file to see its structure, columns, and sample data
      without downloading.",
    "parameters": {
      "type": "object",
      "properties": {
        "dataset_id": {
          "type": "string",
          "description": "Dataset identifier"
        },
        "file_path": {
          "type": "string",
          "description": "Path to file within the dataset"
        }
      },
      "required": ["dataset_id", "file_path"]
    },
  },
  {
    "name": "execute_code",
    "description": "Execute Python code to analyze downloaded files. Use print() to see
      output. pandas, json, pathlib are available. IMPORTANT: Only use files you have
      successfully downloaded - use the exact local_path returned by the download
      tool.",
    "parameters": {
      "type": "object",
      "properties": {
        "code": {
          "type": "string",
          "description": "Python code to execute. Use the exact local_path from
            download results. Do NOT guess file paths."
        }
      },
      "required": ["code"]
    },
  },
  {
    "name": "submit_answer",

```

```

    "description": "Submit your final answer when you have computed the result.",
    "parameters": {
      "type": "object",
      "properties": {
        "answer": {
          "type": "string",
          "description": "The final answer. Format: just the value, e.g., '12345'
                        not '12345 incidents'"
        },
        "reasoning": {
          "type": "string",
          "description": "Brief explanation of how you computed the answer"
        }
      },
      "required": ["answer"]
    }
  }
]

```

G. Annotation Guideline

Each task shall be a question that requires searching over different data sources across Wikipedia/data.gov to answer. In each task.json, each node represents a dataset, and fact is factual information obtained from that dataset, the subquestion is a question constructed by reversing the fact so it be answered via the fact. Finally, the final question shall be answerable by chaining the subquestions together. What you will need to do: Make sure each task involves at least one dataset from data.gov (no all-wikipedia task). If not, mark them and skip the following steps. For each node, verify that the fact and the subquestion are sound and complete, and each fact is grounded in the source file located in S3.. Each fact is of the shape "a set of entities" satisfies "condition", sound means everything in "a set of entities" satisfies "condition", and complete means "a set of entities" are everything satisfies "condition". For example, the fact "Barack Obama" is "the 44th president of USA" is both sound and complete, while the fact "Barack Obama" is "a democratic US president" is sound but not complete, because the subquestion is then who is a democratic US president, whose answer is not deterministic. Pay attention to the wording of the fact, sometimes the fact obtained from a tabular datasets just uses the column name but this could be WRONG, for example 'crime rate' could refer to 'firearm crime rate' or 'violent crime rate' (this information is sometimes in the metadata) but this ambiguity makes the question unanswerable. Pay attention to facts without a time/location qualifier. For those cases, (a) make the language in the question precise and (b) add the metadata file as a node if this information is not self-contained in the tabular dataset. If some nodes are incorrect, mark them and skip the following steps. After verifying each node is correct, the next step is to rewrite the final question. We want questions that looks natural and does not directly hint which data source to search for. The reasoning chain can be viewed as a directed acyclic graph with nodes as vertices (nodes + one for the final question + one for answer) and edges are dependencies of between vertices, i.e. nodes depend on the answer of subquestion from previous nodes, if a node does not depend on any other nodes, it depends on the final question. For this graph, add three parameters. k the number of hops, which is the longest path from final question to answer. n is the number of vertices, and d is the largest number of vertices a vertex depends on.

Some FAQs as examples during annotation: Some answers to subquestions might be incomplete, but the important thing is if it affects the reasoning chain. For example, two subquestions 1. who is Democratics US President and 2. who is US president after 2000, and in a hop, we take an intersection between the two subquestions: who is a Democratics US President after 2000, as long as the answer to the intersection is correct, the task is fine – because eventually each task is consist of the final question and its answer, all subquestion and facts are just for the sake of creating the final question. An excellent question about question chaining: because all we need is the final question, the references like (from the intersection of nodes 1-4) is a place holder for annotators to easily chain questions together. For example, first node has something like Who is the US president born in Hawaii and the second node has something like who is the first lady of ;answer of first node; (please refer to the reasoning-chain field of each task to better understand the logic flow of each task). A derivative of 2 is the problem of skip nodes: we are creating all those subquestions and facts in order to chain the subquestions together, meaning that the answer to previous nodes questions becomes conditions in latter nodes. it is important that the first node's answer is necessary to answer the second node's question. A skipping node is something like who is the first lady of ;Who is the US president born in Hawaii; and is African American because in this case first lady + African American becomes deterministic. We should try to avoid obvious skipping nodes.

H. Example Task Annotation

Listing 2. Complete JSON annotation artifact for a 5-hop LAKEQA task.

```
{
  "question": "What was the 2017-18 total student enrollment in the school district of the city that hosts the land-grant university in the only Eastern Washington county that borders Idaho AND had 2020 census population under 300,000, among counties with school districts achieving ELA performance  $\geq 68\%$  for All Students, All Grades in all four Washington Report Card Assessment releases from 2014-15 through 2017-18?",
  "final_question": "What is the city that hosts the land-grant university in the only Eastern Washington county that borders Idaho and had under 300,000 in 2020? It also consistently performed well on English Language Arts assessments between 2014 and 2018 ( $\geq 68\%$ ). What was the total student enrollment in the school district of that city?",
  "answer": "2941",
  "nodes": {
    "1": {
      "source": "datagov/report-card-assessment-data-2014-15-school-year/files/rows.txt",
      "fact": "In the Washington Report Card Assessment Data 2014-15 release, 21 school districts had ELA PercentMetStandard  $\geq 68\%$  for All Students, All Grades, including districts in King, Clark, Pierce, Whitman, Thurston, and Spokane counties.",
      "subquestion": "Which Washington school districts had ELA performance  $\geq 68\%$  for All Students, All Grades from 2014-15 to 2017-18? (Filter: OrganizationLevel=District; TestSubject=ELA; StudentGroup=All Students; GradeLevel=All Grades; PercentMetStandard $\geq 68$ ).",
      "answer": [
        "Camas School District (Clark)",
        "Lake Washington School District (King)",
        "Dieringer School District (Pierce)",
        "Colfax School District (Whitman)",
        "Mercer Island School District (King)",
        "Griffin School District (Thurston)",
        "Carbonado School District (Pierce)",
        "Freeman School District (Spokane)",
        "Shoreline School District (King)",
        "St. John School District (Whitman)",
        "Issaquah School District (King)",
        "Northshore School District (King)"
      ],
      "sound": false,
      "complete": false,
      "validation_explanation": "The answer should instead be: [ \"Riverside School District\", \"Index Elementary School District\", \"Port Townsend School District\", \"Woodland School District\", \"Deer Park School District\", \"Damman School District\", \"Camas School District\", \"Lake Washington School District\", \"Blaine School District\", \"Chehalis School District\", \"Dieringer School District\", \"Colfax School District\", \"Mercer Island School District\", \"Griffin School District\", \"Carbonado School District\", \"Freeman School District\", \"Starbuck School District\", \"Shoreline School District\", \"St. John School District\", \"Issaquah School District\", \"Northshore School District\"]",
      "revision_subquestion": "Which Washington school districts had ELA performance  $\geq 68\%$  for All Students, All Grades from 2014-15?"
    },
    "2": {
      "source": "datagov/report-card-assessment-data-2015-16-school-year/files/rows.txt",
      "fact": "In the Washington Report Card Assessment Data 2015-16 release, 77 school districts had ELA PercentMetStandard  $\geq 68\%$  for All Students, All Grades, including districts in all 6 counties from the 2014-15 results.",
      "subquestion": "Which Washington school districts had ELA performance  $\geq 68\%$  for All Students, All Grades from 2014-15 to 2017-18? (Filter: OrganizationLevel=District; TestSubject=ELA; StudentGroup=All Students; GradeLevel=All Grades; PercentMetStandard $\geq 68$ ).",
    }
  }
}
```

```

"answer": [
  "Camas School District",
  "Lake Washington School District",
  "Dieringer School District",
  "Colfax School District",
  "Mercer Island School District",
  "Griffin School District",
  "Carbonado School District",
  "Freeman School District",
  "Shoreline School District",
  "St. John School District",
  "Issaquah School District",
  "Northshore School District",
  "and 65 others"
],
"sound": false,
"complete": false,
"validation_explanation": "Answer should not include \"and 65 others\", expand it
  instead: [\"Edmonds School District\", \"Fife School District\", \"West Valley
  School District (Spokane)\", \"Bainbridge Island School District\", \"Sedro-Woolley
  School District\", \"Walla Walla Public Schools\", \"Enumclaw School
  District\", \"Hockinson School District\", \"Snohomish School District\", \"Mercer
  Island School District\", \"Grandview School District\", \"Lake Washington School
  District\", \"Granite Falls School District\", \"Colfax School
  District\", \"Evergreen School District (Stevens)\", \"Camas School
  District\", \"Oak Harbor School District\", \"Issaquah School District\", \"Bellevue
  School District\", \"Clover Park School District\", \"St. John School
  District\", \"Quillayute Valley School District\", \"Almira School
  District\", \"Shoreline School District\", \"South Kitsap School District\", \"Pasco
  School District\", \"Northshore School District\", \"Riverside School
  District\", \"Snoqualmie Valley School District\", \"Tahoma School
  District\", \"Chehalis School District\", \"Yakima School District\", \"Ephrata
  School District\", \"Dieringer School District\", \"North Mason School
  District\", \"University Place School District\", \"Wilbur School
  District\", \"Spokane School District\", \"Central Kitsap School
  District\", \"Olympia School District\", \"Ritzville School District\", \"Port
  Angeles School District\", \"Clarkston School District\", \"Ridgefield School
  District\", \"Anacortes School District\", \"Blaine School District\", \"Garfield
  School District\", \"Riverview School District\", \"Freeman School
  District\", \"Oakesdale School District\", \"Sumner-Bonney Lake School
  District\", \"Colton School District\", \"Damman School District\", \"Wenatchee
  School District\", \"Mead School District\", \"Peninsula School
  District\", \"Everett School District\", \"East Valley School District
  (Spokane)\", \"Lake Stevens School District\", \"Griffin School District\", \"Vashon
  Island School District\", \"Evaline School District\", \"Steilacoom Hist. School
  District\", \"Carbonado School District\", \"Port Townsend School
  District\", \"Index Elementary School District\", \"Pullman School
  District\", \"Rosalia School District\", \"Davenport School District\", \"Lynden
  School District\", \"Liberty School District\", \"Stanwood-Camano School
  District\", \"Tumwater School District\", \"Bellingham School District\", \"Selkirk
  School District\", \"Sequim School District\", \"Sunnyside School District\"]",
"revision_subquestion": "Which Washington school districts had ELA performance >=
  68% for All Students, All Grades from 2015-2016?"
},
"3": {
  "source": "datagov/report-card-assessment-data-2016-17-school-year/files/rows.txt",
  "fact": "In the Washington Report Card Assessment Data 2016-17 release, 67 school
    districts had ELA PercentMetStandard >= 68% for All Students, All Grades,
    including all 14 districts that qualified in 2014-15.",
  "subquestion": "Which Washington school districts had ELA performance >= 68% for All
    Students, All Grades from 2014-15 to 2017-18? (Filter:
    OrganizationLevel=District; TestSubject=ELA; StudentGroup=All Students;
    GradeLevel=All Grades; PercentMetStandard>=68.)",
  "answer": [
    "Camas School District",

```

```

    "Lake Washington School District",
    "Dieringer School District",
    "Colfax School District",
    "Mercer Island School District",
    "Griffin School District",
    "Carbonado School District",
    "Freeman School District",
    "Shoreline School District",
    "St. John School District",
    "Issaquah School District",
    "Northshore School District",
    "and 55 others"
  ],
  "sound": false,
  "complete": false,
  "validation_explanation": "Answer should not be in 2014-2015. Should also not have
    'and 55 others'. It should instead be: [\"Enumclaw School District\", \"Snohomish
    School District\", \"Centralia School District\", \"Mercer Island School
    District\", \"Prosser School District\", \"East Valley School District
    (Spokane)\", \"Clover Park School District\", \"Clarkston School
    District\", \"Bainbridge Island School District\", \"Colfax School
    District\", \"Lake Washington School District\", \"Sequim School
    District\", \"Chehalis School District\", \"Edmonds School District\", \"Camas
    School District\", \"Oakesdale School District\", \"Wenatchee School
    District\", \"Almira School District\", \"Lakewood School District\", \"Sultan
    School District\", \"West Valley School District (Spokane)\", \"Yakima School
    District\", \"Issaquah School District\", \"Tahoma School District\", \"North River
    School District\", \"Spokane School District\", \"Fife School
    District\", \"Grandview School District\", \"Bellevue School District\", \"Dieringer
    School District\", \"Snoqualmie Valley School District\", \"Oak Harbor School
    District\", \"Peninsula School District\", \"Shoreline School
    District\", \"Northshore School District\", \"University Place School
    District\", \"Bickleton School District\", \"Sumner-Bonney Lake School
    District\", \"Carbonado School District\", \"Olympia School District\", \"Damman
    School District\", \"Central Kitsap School District\", \"South Kitsap School
    District\", \"Pullman School District\", \"Quillayute Valley School
    District\", \"St. John School District\", \"Granite Falls School
    District\", \"Spokane International Academy\", \"Anacortes School District\", \"Lake
    Stevens School District\", \"Rosalia School District\", \"Paterson School
    District\", \"Mead School District\", \"Coupeville School District\", \"Wilbur
    School District\", \"Sunnyside School District\", \"Everett School
    District\", \"Riverview School District\", \"Vashon Island School
    District\", \"Freeman School District\", \"Hoquiam School District\", \"LaCrosse
    School District\", \"Davenport School District\", \"Adna School
    District\", \"Griffin School District\", \"Kennewick School District\", \"Medical
    Lake School District\"]",
  "revision_subquestion": "Which Washington school districts had ELA performance >=
    68% for All Students, All Grades from 2016-2017?"
},
"4": {
  "source": "datagov/report-card-assessment-data-2017-18-school-year/files/rows.txt",
  "fact": "In the Washington Report Card Assessment Data 2017-18 release, 56 school
    districts had ELA PercentMetStandard >= 68% for All Students, All Grades,
    including all 13 districts from the 2014-15 intersection.",
  "subquestion": "Which Washington school districts had ELA performance >= 68% for All
    Students, All Grades from 2014-15 to 2017-18? (Filter:
    OrganizationLevel=District; TestSubject=ELA; StudentGroup=All Students;
    GradeLevel=All Grades; PercentMetStandard>=68.)",
  "answer": [
    "Camas School District",
    "Lake Washington School District",
    "Dieringer School District",
    "Colfax School District",
    "Mercer Island School District",
    "Griffin School District",

```

```

    "Carbonado School District",
    "Freeman School District",
    "Shoreline School District",
    "St. John School District",
    "Issaquah School District",
    "Northshore School District",
    "and 44 others"
  ],
  "sound": false,
  "complete": false,
  "validation_explanation": "Should not include 44 others. The answer should be:
    [\"University Place School District\", \"Enumclaw School District\", \"Mercer
    Island School District\", \"Bainbridge Island School District\", \"Lake Washington
    School District\", \"Camas School District\", \"Sunnyside School
    District\", \"Colfax School District\", \"Issaquah School District\", \"Blaine
    School District\", \"Pullman School District\", \"Snoqualmie Valley School
    District\", \"Bellevue School District\", \"Carbonado School
    District\", \"Sumner-Bonney Lake School District\", \"Northshore School
    District\", \"Dieringer School District\", \"Colton School District\", \"Almira
    School District\", \"Evergreen School District (Stevens)\", \"Sedro-Woolley School
    District\", \"Shoreline School District\", \"Garfield School District\", \"Wilbur
    School District\", \"Tahoma School District\", \"Olympia School District\", \"St.
    John School District\", \"Freeman School District\", \"Snohomish School
    District\", \"Palouse School District\", \"Davenport School District\", \"Central
    Kitsap School District\", \"Oakesdale School District\", \"Peninsula School
    District\", \"Grapeview School District\", \"Riverview School
    District\", \"Coulee-Hartline School District\", \"Vashon Island School
    District\", \"Ridgefield School District\", \"LaCrosse School
    District\", \"Anacortes School District\", \"Steilacoom Hist. School
    District\", \"Everett School District\", \"Coupeville School District\", \"Griffin
    School District\", \"Centralia School District\", \"Spokane School
    District\", \"Mead School District\", \"Edmonds School District\", \"Stanwood-Camano
    School District\", \"Fife School District\", \"Seattle School District No.
    1\", \"Lake Stevens School District\", \"Endicott School District\", \"Tumwater
    School District\", \"Selkirk School District\"]",
  "revision_subquestion": "Which Washington school districts had ELA performance >=
    68% for All Students, All Grades from 2017-2018?"
},
"5": {
  "source": "wikipedia/Eastern_Washington/content.txt",
  "fact": "The Eastern Washington article lists Whitman and Spokane among the counties
    in Eastern Washington.",
  "subquestion": "Among counties from <hop 1 answer: King, Clark, Pierce, Whitman,
    Thurston, Spokane>, which are in Eastern Washington?",
  "answer": "Whitman and Spokane",
  "sound": false,
  "complete": false,
  "validation_explanation": "Data is true and accurate. However, there seems to be a
    missing node from node 4 to node 5. The intersection of all nodes in node 4 are
    still 12 districts. Yet, they somehow all transform into the 6 counties that
    contain those 12 districts any sources.",
  "revision_subquestion": "Add nodes 4 and 5 to convert the districts into counties.
    Here are the districts: Camas School DistrictCarbonado School DistrictColfax
    School DistrictDieringer School DistrictFreeman School DistrictGriffin School
    DistrictIssaquah School DistrictLake Washington School DistrictMercer Island
    School DistrictNorthshore School DistrictShoreline School DistrictSt. John School
    District"
},
"6": {
  "source": "wikipedia/Whitman_County,_Washington/content.txt",
  "fact": "According to the Wikipedia article for Whitman County, Washington,
    'Adjacent counties' include 'Benewah County, Idaho - northeast', 'Latah County,
    Idaho - east', and 'Nez Perce County, Idaho - southeast', confirming Whitman
    County borders Idaho. 'As of the 2020 census, the population was 47,973.'",

```

```

    "subquestion": "Among counties from <hop 2 answer: Whitman, Spokane>, which county
      borders Idaho AND has 2020 census population under 300,000?",
    "answer": "Whitman County: borders Idaho (YES, per adjacent counties list),
      population 47,973 (YES, under 300,000) $\to$ MEETS BOTH CRITERIA",
    "sound": true,
    "complete": true,
    "validation_explanation": "Population 47,973 (line 1), borders with Idaho line 48-50"
  },
  "7": {
    "source": "wikipedia/Spokane_County,_Washington/content.txt",
    "fact": "According to the Wikipedia article for Spokane County, Washington,
      'Adjacent counties' include 'Bonner County, Idaho - northeast', 'Kootenai County,
      Idaho - east', and 'Benewah County, Idaho - southeast', confirming Spokane County
      borders Idaho. The 2020 census population was 539,339.",
    "subquestion": "Among counties from <hop 2 answer: Whitman, Spokane>, which county
      borders Idaho AND has 2020 census population under 300,000?",
    "answer": "Spokane County: borders Idaho (YES, per adjacent counties list),
      population 539,339 (NO, exceeds 300,000) $\to$ DOES NOT MEET BOTH CRITERIA",
    "sound": true,
    "complete": true,
    "validation_explanation": "population was 539,339 (line 1). Borders idaho (lines
      80-82)"
  },
  "8": {
    "source": "wikipedia/Pullman,_Washington/content.txt",
    "fact": "According to the Wikipedia article for Pullman, Washington, Pullman is the
      most populous city in Whitman County and is home to Washington State University,
      a public research land-grant university.",
    "subquestion": "Which city hosts the land-grant university in <hop 3 answer: Whitman
      County>?",
    "answer": "Pullman hosts the land-grant university (WSU) in Whitman County",
    "sound": true,
    "complete": true,
    "validation_explanation": "Line 3"
  },
  "9": {
    "source": "datagov/report-card-enrollment-2017-18-school-year/files/rows.txt",
    "fact": "According to the Washington Report Card Enrollment Data 2017-18, Pullman
      School District (in Whitman County) had 2,941 total students enrolled for All
      Grades.",
    "subquestion": "What is the 2017-18 total student enrollment for the school district
      in <hop 4 answer: Pullman>? (Filter: OrganizationLevel=District;
      DistrictName=Pullman School District; GradeLevel=All Grades; column=All
      Students.)",
    "answer": "2941",
    "sound": true,
    "complete": true,
    "validation_explanation": "True and accurate by query."
  }
},
"reasoning_chain": [
  "HOP 1 (d=4, WA Report Card Assessment Data 2014-15 through 2017-18 - nodes 1-4):",
  " Node 1: Districts with ELA >= 68% in 2014-15 $\to$ 21 districts in 6 counties",
  " Node 2: Districts with ELA >= 68% in 2015-16 $\to$ 77 districts",
  " Node 3: Districts with ELA >= 68% in 2016-17 $\to$ 67 districts",
  " Node 4: Districts with ELA >= 68% in 2017-18 $\to$ 56 districts",
  " Intersection: 12 districts in 6 counties (King, Clark, Pierce, Whitman, Thurston,
    Spokane)",
  "",
  "HOP 2 (d=1, Wikipedia Eastern Washington - node 5):",
  " Node 5 (Wikipedia Eastern Washington): Whitman and Spokane are Eastern Washington
    counties",
  " Result: Whitman and Spokane are the Eastern Washington counties from hop 1",
  "",
  "HOP 3 (d=2, Wikipedia county pages - nodes 6-7):",

```

```
" Node 6 (Wikipedia Whitman_County): Borders Idaho (YES), pop 47,973 $\to$ MEETS BOTH
  CRITERIA",
" Node 7 (Wikipedia Spokane_County): Borders Idaho (YES), pop 539,339 $\to$ DOES NOT
  MEET (pop > 300k)",
" Result: Whitman County (REQUIRES hop 2 to know which counties to check)",
"",
"HOP 4 (d=1, Wikipedia Pullman - node 8):",
" Node 8 (Wikipedia Pullman): Pullman is home to WSU (land-grant) in Whitman County",
" Result: Pullman hosts the land-grant university (REQUIRES hop 3)",
"",
"HOP 5 (d=1, data.gov enrollment - node 9):",
" Node 9 (WA Report Card Enrollment 2017-18): Pullman School District = 2,941
  students",
" Result: 2941 (REQUIRES hop 4 to know the city is Pullman)",
"",
"Final answer: 2941"
],
"annotated_by": "[annotator]",
"annotation_timestamp": "[timestamp]"
}
```